Step 1: Install Required Libraries

What are we performing?

- Installing the required LangChain packages.

Why are we performing it?

- These libraries are required to build the RAG pipeline.

Difference from Previous Notebook

- This notebook uses LangChain's FAISS implementation instead of manually creating a FAISS index.

In [ ]:
```
# pip install langchain_huggingface
# !pip install langchain_community
```

Step 2: Import Required Libraries

What are we performing?

- Importing libraries for text preprocessing, chunking, embeddings and vector database.

Why are we performing it?

- These libraries are used throughout the RAG pipeline.

Difference from Previous Notebook

- `HuggingFaceEmbeddings` and `FAISS` from LangChain are used instead of manual embedding generation and FAISS indexing.

In [2]:
```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from sentence_transformers import SentenceTransformer
from langchain_community.vectorstores import FAISS
from langchain_huggingface import HuggingFaceEmbeddings
import spacy
import faiss
```

```
C:\Users\LENOVO\AppData\Local\Temp\ipykernel_16892\535571621.py
:3: DeprecationWarning: `langchain-community` is being sunset
and is no longer actively maintained. See
https://github.com/langchain-ai/langchain-community/issues/674
for details and migration guidance toward standalone
integration packages.
  from langchain_community.vectorstores import FAISS
```

Step 3: Load the Document

What are we performing?

- Reading the text file.
- Storing it in the variable `data`.

Why are we performing it?

- This document acts as the knowledge source for the RAG system.

In [3]:
```
data = open(r'machine_learning_2000_sentences.txt').read()
```

$$ Text Normalization $$

What are we performing?

- Preparing the text for preprocessing.

Why are we performing it?

- Clean text improves chunk quality and embedding quality.

Step 4: Convert Text to Lowercase

What are we performing?

- Converting all characters to lowercase.

Why are we performing it?

- Creates a consistent text format.

In [4]:
```
data = data.lower()
```

Step 5: Remove Extra Spaces

What are we performing?

- Removing multiple spaces using Regex.

Why are we performing it?

- Produces cleaner text.

In [5]:
```
import re
data = re.sub(r'\s{2,}',' ', data)
```

Step 6: Remove Statement Labels

What are we performing?

- Removing patterns like `Machine Learning Statement 1:` .

Why are we performing it?

- These labels do not contribute to the meaning of the document.

In [6]:
```
data = re.sub(r'machine learning statement \d+\:','',data)
```

Step 7: Expand Contractions

What are we performing?

- Importing the contractions library.
- Replacing contractions with their full form.

Why are we performing it?

- Used to expand words such as `can't → cannot` .
- Improves text consistency.

In [7]:
```
import contractions
data = contractions.fix(data)
```

Step 8: Remove Punctuation

What are we performing?

- Importing punctuation symbols.
- Removing punctuation using Regex.

Why are we performing it?

- Keeps only meaningful words.

In [8]:
```
data = re.sub(r'[^0-9a-zA-Z\s]','',data)
```

Step 12: Spell Correction

What are we performing?

- Importing TextBlob.

Why are we performing it?

- Used for correcting spelling mistakes if required.

In [ ]:
```
# from textblob import TextBlob
# str(TextBlob(data).correct())
```

Step 13: Lemmatization using spaCy

What are we performing?

- Importing spaCy.

Why are we performing it?

- Used for tokenization, stop-word removal and lemmatization.

In [9]:
```
nlp = spacy.load('en_core_web_sm')
```

Step 14: Tokenize and Lemmatize

What are we performing?

- Splitting text into tokens.
- Removing stop words.
- Converting words to their root form.

Why are we performing it?

- Produces cleaner text for embedding generation.

In [10]:
```
tokens = nlp(data)
updated_tokens = [token.lemma_ for token in tokens if not token.is_stop]
```

Step 15: Convert Tokens Back to Text

What are we performing?

- Joining processed tokens into a single string.

Why are we performing it?

- Creates the final cleaned document.

In [11]:
```
data = ' '.join(updated_tokens).strip()
```

Step 16: Chunk the Document

What are we performing?

- Creating a text splitter.

Why are we performing it?

- Splits the document into smaller chunks for retrieval.

In [12]:
```
splitter = RecursiveCharacterTextSplitter(
    chunk_size = 100, chunk_overlap = 20
)
```

Step 14: Create Document Chunks

What are we performing?

- Creating LangChain `Document` objects.

Why are we performing it?

- LangChain stores both content and metadata.

Difference from Previous Notebook

- Previous notebook used `split_text()` .
- This notebook uses `create_documents()` , which returns `Document` objects.

In [13]:
```
chunks = splitter.create_documents([data])
```

Step 15: Display the First Chunk

What are we performing?

- Printing the first chunk.

Why are we performing it?

- To verify chunk creation.

In [16]:
```
chunks[0]
```

Out[16]:
```
Document(metadata={}, page_content='workflow emphasizing
unsupervise \n learning improve carefully validate feature scale')
```

Step 16: Check the Chunk Type

What are we performing?

- Checking the datatype of a chunk.

Why are we performing it?

- Confirms that each chunk is a LangChain `Document` .

In [18]:
```
type(chunks[0])
```

Out[18]:
```
langchain_core.documents.base.Document
```

Step 17: Display Chunk Content

What are we performing?

- Printing only the text inside the chunk.

Why are we performing it?

- To inspect the actual chunk content.

In [19]:
```
print(chunks[0].page_content)
```

```
workflow emphasizing unsupervise
 learning improve carefully validate feature scale
```

Step 18: Add Metadata to Chunks

What are we performing?

- Adding metadata to each document chunk.

Why are we performing it?

- Helps identify the source of the retrieved chunk.
- Useful when retrieving data from multiple documents.

In [22]:
```
chunks[0].metadata = 'machine_learning_2000_sentences.txt'
```

In [23]:
```
chunks[0]
```

Out[23]:
```
Document(metadata='machine_learning_2000_sentences.txt',
page_content='workflow emphasizing unsupervise \n learning improve
carefully validate feature scale')
```

In [24]:
```
chunks[0].metadata = {'file_name':'machine_learning_2000_sentences.txt'}
```

In [25]:
```
chunks[0]
```

Out[25]:
```
Document(metadata={'file_name':
'machine_learning_2000_sentences.txt'}, page_content='workflow
emphasizing unsupervise \n learning improve carefully validate
feature scale')
```

Step 19: Load the Embedding Model

What are we performing?

- Loading the Hugging Face embedding model.

Why are we performing it?

- Converts document chunks into vector embeddings for semantic search.

Difference from Previous Notebook

Previous notebook:
Used SentenceTransformer().encode()

This notebook:
Uses LangChain's HuggingFaceEmbeddings.

In [26]:
```
embedding_model = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-mpnet-base-v2"
)
```

```
Loading weights: 100%|██████████| 199/199 [00:00<00:00,
1188.92it/s]
```

Step 20: Create the LangChain FAISS Vector Store

What are we performing?

- Creating a FAISS vector database from the document chunks.

Why are we performing it?

- Stores embeddings efficiently.
- Enables semantic similarity search.

Difference from Previous Notebook

Previous notebook:
Manually generated embeddings.
Manually created the FAISS index.

> Syntax:

`python
chunk_embeddings = embedding_model.encode(chunks).astype('float32')
dimension = chunk_embeddings.shape[1]
faiss.normalize_L2(chunk_embeddings)
index_faiss_db = faiss.IndexFlatIP(dimension)
index_faiss_db.add(chunk_embeddings)
`

This notebook:
LangChain performs both operations automatically.

In [27]:
```
vector_db = FAISS.from_documents(
    documents=chunks,
    embedding=embedding_model
)
```

In [28]:
```
vector_db
```

Out[28]:
```
<langchain_community.vectorstores.faiss.FAISS at 0x240814017f0>
```

Step 21: Perform Similarity Search

What are we performing?

- Searching the vector database using the user's query.

Why are we performing it?

- Retrieves the most relevant chunks based on semantic similarity.

Difference from Previous Notebook

Previous notebook: Used `index_faiss_db.search()` .
This notebook: Uses `vector_db.similarity_search()` .

In [29]:
```
user_query = "What is Machine Learning?"
r_chunks = vector_db.similarity_search(user_query)
```

Step 22: Display Retrieved Chunks

What are we performing?

- Printing every retrieved chunk.

Why are we performing it?

- To verify that the retrieved information is relevant.

In [30]:
```
for chunk in r_chunks:
    print(chunk.page_content)
```

```
supervise learning improve carefully validate supervised
supervise learning improve carefully validate supervised
supervise learning improve carefully validate supervised
supervise learning improve carefully validate supervised
```

Step 23: Merge Retrieved Chunks

What are we performing?

- Combining all retrieved chunks into one text.

Why are we performing it?

- Creates the context that will be sent to the LLM.

In [ ]:
```
updated_r_chunks = set()
for chunk in r_chunks:
    updated_r_chunks.add(chunk.page_content)

R_Text = '\n'.join(updated_r_chunks)
```

In [32]:
```
R_Text
```

Out[32]: 'supervise learning improve carefully validate supervised'

In [41]:

```
import os
import requests

API_URL = "https://router.huggingface.co/v1/chat/completions"

headers = {
    "Authorization": f"Bearer {os.environ['HF_TOKEN']}",
}


def generate_response(user_query):
    # ---------------------- Retrieval ----------------------
    r_chunks = vector_db.similarity_search(user_query)

    context = "\n".join(
        list(set(chunk.page_content for chunk in r_chunks))
    )

    # ---------------------- Prompt ----------------------
    prompt = f"""
You are a helpful AI assistant.

Answer the user's question only using the provided context.

If the answer is not available in the context, reply:
"I couldn't find the answer in the provided context."

Context:
{context}

Question:
{user_query}

Answer:
"""

    payload = {
        "model": "Qwen/Qwen3-0.6B:featherless-ai",
        "messages": [
            {
                "role": "user",
                "content": prompt
            }
        ]
    }

    response = requests.post(
        API_URL,
        headers=headers,
        json=payload,
        timeout=90
    )

    result = response.json()

    return result["choices"][0]["message"]["content"]
```

```
user_query = "Explain Machine Learning"
answer = generate_response(user_query)
print(answer)
```

```
<think>
Okay, the user is asking to explain Machine Learning. Let me start
by recalling the context provided. The context mentions learning
statement 1898 workflow emphasizing overfitting, improving
carefully validate unsupervised learning, and workflow emphasizing
supervision.

First, I need to make sure I understand what the context is about.
It seems like there's a section related to machine learning
concepts, possibly from a course or a textbook. The key points
here are about overfitting, supervised learning, and feature
scaling.

The user wants an explanation of Machine Learning. I should start
by defining what Machine Learning is. Then, mention supervised
learning as a common approach, where there's supervised learning
and unsupervised learning. Also, feature scaling is important for
improving performance.

I need to check if the context includes all necessary elements.
The context does talk about improving and validating unsupervised
learning and supervision. So the answer should cover these points.

Wait, the user's question is straightforward, so maybe just list
the main components without getting too technical. Make sure the
answer is clear and covers the essentials. Also, ensure that the
answer is in the context provided, so I shouldn't add anything not
mentioned there.
</think>

Machine Learning is a subset of artificial intelligence that
enables systems to learn patterns from data. It involves
algorithms that analyze data to make predictions or decisions with
minimal human intervention. Key concepts include supervised
learning (where there is labeled data), unsupervised learning
(where there is unlabeled data), and feature scaling to improve
performance. The context highlights the importance of improving
and validating both supervised and unsupervised learning
processes.
```

In [ ]: